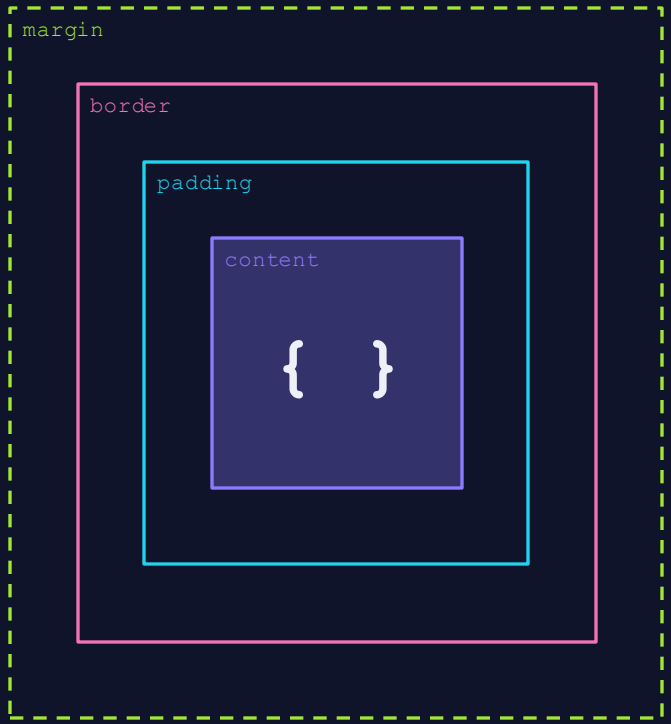
WEB PROGRAMLAMA I · MDN ÇEKİRDEK MODÜLLER

# CSS ile Stil ve Düzen

CSS styling basics
CSS text styling
CSS layout

KCŞ A.Ş. | Çekirdek modüller: 2, 3 ve 4. başlıklar



```
@import url(bugun.css);
```

# Bugünün yolculuğu: 3 modül, 26 ders

## 01

15 ders

### CSS temelleri

CSS styling basics

sözdizimi, seçiciler, box model, cascade, birimler, boyut, arka plan, taşma, form/tablo, debug

## 02

4 ders

### Metin stili

CSS text styling

font ve metin temelleri, listeler, linkler, web fontları

## 03

7 ders

### CSS düzeni

CSS layout

normal akış, float, positioning, flexbox, grid, responsive, media queries

≈ **Gündelik hayatta:** HTML evin iskeletiyse CSS iç mimarıdır: önce boya ve mobilya (temeller), sonra yazılar ve tabelalar (metin), en son odaların yerleşimi (düzen).

/* 1994 → bugün */

# CSS'in kısa tarihi

## 1994

Håkon Wium Lie, CERN'de CSS'i önerir: içerik HTML'de, görünüm ayrı bir dilde.

## 1996

CSS1 W3C önerisi olur: font, renk, metin hizalama, margin, border, padding.

## 1998

CSS2: konumlandırma (position), z-index, media türleri.

## 2011

CSS 2.1 resmîleşir; CSS3 artık tek dosya değil, ayrı modüllerden oluşur.

## 2010'lar

Flexbox ve media queries yaygınlaşır; 2017'de tarayıcılar Grid'i destekler.

## 2020'ler

:has(), container queries, cascade layers ve iç içe (nested) CSS gelir.

≈ **Gündelik hayatta:** CSS tek seferde basılmış bir ansiklopedi değil, cilt cilt çıkan bir seri: her modül kendi hızında güncellenir.

01

# CSS temelleri

CSS styling basics

15 ders

- What is CSS?
- Getting started
- Basic selectors
- Attribute selectors
- Pseudo-classes & elements
- Combinators
- Box model
- Handling conflicts
- Values and units
- Sizing
- Backgrounds and borders
- Overflow
- Images, media, forms
- Styling tables
- Debugging CSS

h1 { color: red; }

# CSS nedir? Bir kuralın anatomisi

styles.css

```
h1 {
  color: red;
  font-size: 2.5em;
}
```

seçici: h1 | özellik: color | değer: red | bildirim bloğu: { … }

Kural tabanlı bir dil: hangi öğeyi (seçici), neyini (özellik), nasıl (değer) değiştireceğini söylersin. Her özellik sadece belirli değer türlerini kabul eder.

### Tarayıcının varsayılan stilleri

Hiç CSS yazmasan da başlıklar büyük, linkler mavi ve altı çizili görünür. Bunlar HTML'den değil, tarayıcıya gömülü varsayılan stil sayfasından gelir.

### Tarayıcı CSS'i nasıl uygular?

1. HTML → DOM ağacı
2. Kurallar seçicilerine göre öğelere dağıtılır → render ağacı
3. Render ağacı ekrana boyanır (painting)

≈ **Gündelik hayatta:** HTML bir evin odalarıysa, CSS iç mimarın listesidir.

<link rel="stylesheet">

# CSS'i sayfaya eklemenin 3 yolu

## Harici dosya
**En yaygın, en iyi**

```
<link rel="stylesheet"
      href="styles.css" />
```

Bir dosya yüzlerce sayfayı stiller. Yol mantığı resimlerle aynı: styles/style.css, ../styles/style.css.

## Dahili <style>
**Bazen gerekli**

```
<style>
  p { color: purple; }
</style>
```

Head içine yazılır. Çok sayfalı sitede her sayfaya tekrar yazmak gerekir; dosyaya erişemediğin CMS'lerde işe yarar.

## Satır içi style
**Kaçınılmalı**

```
<span style="color: purple;
  font-weight: bold">…</span>
```

Tek öğeyi etkiler. Bakımı en zor yöntem: görünüm ile içerik iç içe girer. E-posta şablonları gibi zorunlu durumlar dışında kullanma.

calc()  @media  /* */

# Söz diziminin diğer parçaları

styles.css

```css
/* fonksiyonlar */
.box {
  width: calc(90% - 30px);
  transform: rotate(0.8turn);
}
/* @kural: koşullu CSS */
@media (width >= 30em) {
  body { background-color: blue; }
}
/* kısayol: üst sağ alt sol */
p { padding: 10px 15px 15px 5px; }
```

**Fonksiyonlar:** calc() CSS içinde hesap yapar; rotate() gibi dönüşüm fonksiyonları transform'a verilir.

**@kurallar:** CSS'e talimat verir. En yaygını @media: koşul sağlanınca içindeki kurallar uygulanır.

**Kısayollar:** padding, margin, border, font, background tek satırda birden çok değeri ayarlar.

**Yorumlar:** /* … */ ile yazılır; bölüm başlığı koymak ya da bir kuralı geçici kapatmak için kullanılır.

**Boşluk:** Fazla boşluk yok sayılır, ama değerler arasında bir boşluk şart: 0auto geçersiz, 0 auto geçerli.

≈ **Gündelik hayatta:** kısayol özellikler kısaltmalı adres yazmak gibi: “Atatürk Cd. No:5 D:3” tek satırda dört bilgiyi taşır.

p .highlight #one *

# Temel seçiciler

| | | |
|---|---|---|
| **Tür seçici** | `h1 { … }` | o türdeki tüm öğeleri seçer |
| **Sınıf seçici** | `.highlight { … }` | class'ı taşıyan her öğe; bir öğede birden çok sınıf olabilir |
| **Zincir sınıf** | `.notebox.danger { … }` | iki sınıfa birden sahip olanlar (boşluksuz yazılır) |
| **ID seçici** | `#one { … }` | tek bir öğe; bir id sayfada bir kez kullanılır |
| **Seçici listesi** | `h1, .special { … }` | aynı stili birden çok seçiciye virgülle ver |
| **Evrensel seçici** | `* { margin: 0; }` | her öğe; sıfırlama (reset) dosyalarında görülür |

≈ **Gündelik hayatta:** okulda anons yapmak gibi: “tüm öğrenciler” (tür), “futbol takımı” (sınıf), “Ahmet Yılmaz” (id). Listede tek seçici hatalıysa bütün kural yok sayılır!

display: block | inline | inline-block

# Box model 1: blok ve satır içi kutular

## Blok (block)

- yeni satırda başlar
- width ve height'a uyar
- padding, margin, border diğerlerini iter
- genişlik verilmezse kabın tamamını doldurur
- ör: h1, p

## Satır içi (inline)

- yeni satıra geçmez
- width, height ve üst/alt margin etkisiz
- üst/alt padding görünür ama komşuları itmez, üstüne biner
- sol/sağ boşluklar komşuları iter
- ör: a, span, em

## inline-block

İkisinin karışımı: yeni satıra geçmez ama width/height'a uyar ve padding ile komşularını iter. Navigasyon linklerine geniş tıklama alanı vermek için ideal.

```
display: flex;
/* dış: block
   iç: flex */
```

≈ **Gündelik hayatta:** blok öğeler rafa tek tek dizilen kitaplar, satır içi öğeler cümlenin içindeki kelimeler gibidir.

box-sizing: border-box;

# Box model 2: kutunun katmanları




```
.box {
  width: 350px;
  height: 150px;
  padding: 25px;
  border: 5px solid; }
```

### Standart

**410 × 210 px**

width sadece içeriği ölçer; padding ve border eklenir.

### border-box

**350 × 150 px**

width görünen kutunun tamamıdır; hesap yapmak gerekmez.

### Margin çökmesi

Dikeyde değen iki pozitif margin birleşir, büyük olan kalır: 50px + 30px = 50px. Margin negatif olabilir, padding olamaz.

≈ **Gündelik hayatta:** kargo kutusu: ürün (content), köpük dolgu (padding), karton (border), raftaki diğer kutulardan uzaklık (margin).

# Çakışmalar 1: hangi kural kazanır?

## Cascade (basamak)

```
h1 { color: red; }
h1 { color: blue; }
```

Eşit güçteki kurallarda sonra yazılan kazanır → mavi.

## Özgüllük

```
.main-heading { color: red; }
h1 { color: blue; }
```

Daha isabetli seçici kazanır: sınıf > tür → kırmızı.

## Kalıtım

```
body { color: blue; }
span { color: black; }
```

color, font-family gibi özellikler çocuklara geçer; width, margin, border geçmez.

≈ **Gündelik hayatta:** ailede kural: son söyleyen geçerli (cascade), müdür veliden üstün (özgüllük), soyadı çocuğa geçer ama ayakkabı numarası geçmez (kalıtım).

1-0-0 > 0-99-99

# Çakışmalar 2: özgüllük puanı

| Seçici | ID | Sınıf | Öğe | Puan |
|---|---|---|---|---|
| `h1` | 0 | 0 | 1 | 0-0-1 |
| `h1 + p::first-letter` | 0 | 0 | 3 | 0-0-3 |
| `li > a[href*="en"] > .warn` | 0 | 2 | 2 | 0-2-2 |
| `#identifier` | 1 | 0 | 0 | 1-0-0 |

ID = 100, sınıf / öznitelik / sözde sınıf = 10, öğe / sözde öğe = 1. Sütunlar bağımsızdır: milyon sınıf bir ID'yi yenemez. *, birleştiriciler ve :where() puan eklemez.

**Öncelik sırası (düşük → yüksek)**

1. Tarayıcı varsayılanı
2. Kullanıcı stili
3. Geliştirici (senin CSS'in)
4. Satır içi style (≈ 1-0-0-0)
5. !important: hepsini ezer; mecbur kalmadıkça kullanma, hata ayıklamayı zorlaştırır

≈ **Gündelik hayatta:** futbolda puan tablosu: önce galibiyete (ID) bakılır, eşitse beraberliğe (sınıf), o da eşitse atılan gole (öğe).

px em rem % vw

# Değerler ve birimler 1: uzunluklar

## Mutlak birimler

Her yerde aynı boyut sayılır. cm, mm, in, pt, pc çoğunlukla baskı içindir; ekranda en çok px kullanılır. 1in = 96px. 1px her zaman tek bir fiziksel piksel değildir.

```
.px { width: 200px; }
.vw { width: 10vw; }
```

## em ve rem

em: font-size'da ebeveynin, diğer özelliklerde öğenin kendi font boyutu. İç içe listelerde 1.3em büyüye büyüye gider. rem: her zaman kök (html) font boyutu; iç içe olunca büyümez.

```
.ems li { font-size: 1.3em; }
.rems li { font-size: 1.3rem; }
```

## Yüzde ve sayı

% her zaman başka bir değere göredir: width'te ebeveyn genişliğine, font-size'da ebeveyn fontuna. 400px kutuda 40% = 160px. Bazı özellikler birimsiz sayı alır: opacity 0–1.

```
.percent { width: 40%; }
.box { opacity: 0.6; }
```

≈ **Gündelik hayatta:** em annenin ayakkabısına göre numara seçmek, rem ise herkesin aynı standart kalıba göre seçmesi gibidir.

`rgb(197 93 161 / .7)`

# Değerler ve birimler 2: renk ve diğerleri

| | |
|---|---|
| `antiquewhite` | anahtar kelime |
| `#c55da1` | onaltılık (hex) |
| `rgb(2 121 139)` | rgb() |
| `hsl(174 77% 31%)` | hsl(): ton, doygunluk, açıklık |

Alfa (şeffaflık) / ile eklenir: rgb(2 121 139 / 0.3). Alfa sadece rengi saydamlaştırır; opacity ise öğeyi içeriğiyle birlikte. Projede tek bir renk yazımı seç.

## Diğer değer türleri

**<image>:** url("star.png") ya da linear-gradient(…)

**<position>:** background-position: right 60px; (önce yatay, sonra dikey)

**Tanımlayıcı / metin:** red tırnaksız anahtar kelime; content: "…" tırnaklı metin

**Fonksiyonlar:** translate(), rotate(), scale(); matematik için calc(20% + 100px), min(), max(), clamp()

≈ **Gündelik hayatta:** boya kataloğu gibi: rengi adıyla (“bordo”), kod numarasıyla (#c55da1) ya da karışım oranıyla (rgb) isteyebilirsin.

min-height  max-width  vh

# Boyutlandırma

**Doğal (intrinsic) boyut:** CSS'ten önceki boyut. Resim dosyasından gelir; boş bir div'in yüksekliği yoktur.

**Dış (extrinsic) boyut:** width/height verirsen içerik sığmak zorunda; sığmazsa taşar. Sabit yükseklik riskli.

**Yüzde:** kapsayan bloğa göredir. Dikkat: % margin ve padding hep genişliğe göre hesaplanır.

**min-height:** en az bu kadar ol, içerik artarsa büyü. Değişken içerikte height yerine kullan.

**max-width: 100%:** resim kaba sığana kadar küçülür ama doğal boyutundan büyümez.

**vw / vh:** ekranın genişlik/yüksekliğinin %1'i. 100vh tam ekran bir banner yapar.

sizing.css

```css
.box {
  min-height: 100px;
  width: 200px;
}
img { max-width: 100%; }
.tam { height: 100vh; }
```

**Uyarı**

Büyük resmi max-width ile tarayıcıda küçültme: indirme yine büyük olur. Uygun boyutta resim sun.

≈ **Gündelik hayatta:** min-height esnek pantolon kemeri gibi: en az bu bel ölçüsü, gerekirse genişler. Sabit height ise dar bir kot.

background: … / cover no-repeat

# Arka planlar ve kenarlıklar

backgrounds.css

```css
.box {
  background-image: url("star.png");
  background-repeat: no-repeat;
  background-size: cover;
  background-position: top 20px right 10px;
  border: 5px solid #0b385f;
  border-radius: 1em;
}
```

Birden çok resim virgülle verilir; listedeki son resim en altta kalır. background kısayolunda renk sadece son virgülden sonra, size ise position'dan hemen sonra / ile yazılır: center/80%.

**background-repeat:** no-repeat, repeat-x, repeat-y, space, round

**background-size:** cover kutuyu doldurur (taşan kırpılır), contain sığdırır (boşluk kalabilir)

**background-attachment:** scroll, fixed (ekrana sabit), local (içerikle kayar)

**Gradyan:** linear-gradient(105deg, …) bir resim gibi background-image'a verilir

**Kenarlık:** border kısayolu ya da border-top, border-width… ; border-radius ile köşe yuvarlat

**Erişilebilirlik:** metinle zemin arasında kontrast olsun; resim yüklenmezse diye background-color da ver

≈ **Gündelik hayatta:** duvar kağıdı döşemek: tekrar mı etsin (repeat), tüm duvarı mı kaplasın (cover), çerçeve kalın mı ince mi (border).

overflow: clip | auto | scroll | hidden

# Taşma (overflow)

## CSS veri kaybından kaçınır

Kutu içeriğe dar gelirse CSS içeriği gizlemez, taşırır. Gizlenen içerik fark edilmeyebilir (ör. formun gönder butonu kaybolur); görünen taşma ise sorunu hemen gösterir. width/height verdiysen taşmayı yönetmek senin sorumluluğun.

```
.box {
  height: 100px;
  overflow: auto; }
```

| Değer | Davranış |
|---|---|
| visible | varsayılan: taşan görünür |
| clip | keser, asla görünmez |
| auto | gerekirse kaydırma çubuğu |
| scroll | her zaman kaydırma çubuğu |
| hidden | keser ama kod/Tab ile kaydırılabilir |

overflow-x / overflow-y eksenleri ayrı ayarlar. En sağlam yol: sabit yükseklikten kaçınmak, az ve çok içerikle test etmek.

≈ **Gündelik hayatta:** ağzına kadar dolu bavul: fermuarı zorla kapatıp eşyaları ezebilirsin (clip) ya da yanına küçük bir çanta eklersin (auto).

object-fit: cover;

# Resimler, medya ve form öğeleri

## Yerine geçen (replaced) öğeler

img ve video'nun iç düzenini CSS değiştiremez; doğal boyut ve en-boy oranları vardır. max-width: 100% onları kaba sığdırır.

```
img { width: 100%; height: 100%; }
.cover   { object-fit: cover; }
.contain { object-fit: contain; }
```

cover kutuyu doldurur ve kırpar, contain resmin tamamını gösterir. object-fit için img'in kendisi boyutlandırılmış olmalı. Grid içinde resimler esnemez, alanın başına hizalanır.

## Formlar

Metin kutuları (text, email, textarea) normal kutu gibi stillenir. Karmaşık girişler işletim sistemince çizilir, stillemek zordur. Tutarlı başlangıç için:

```
button, input, select, textarea {
  font-family: inherit;
  font-size: 100%;
  box-sizing: border-box;
  padding: 0; margin: 0; }
```

Form kontrolleri form kontrolü gibi görünmeye devam etmeli.

≈ **Gündelik hayatta:** object-fit, fotoğrafı çerçeveye yerleştirmek gibi: taşanı kesersin (cover) ya da kenarlarda boşluk bırakırsın (contain).

`tbody tr:nth-child(odd)`

# Tabloları stillemek

tables.css

```css
table {
  table-layout: fixed;
  width: 90%;
  border-collapse: collapse;
}
th, td { padding: 0.3em; }
tr :nth-child(2) {
  text-align: right; }
tbody tr:nth-child(odd) {
  background-color: #dddddd; }
caption { caption-side: bottom; }
```

**table-layout: fixed:** sütun genişlikleri içeriğe değil başlıklara göre, öngörülebilir olur.

**border-collapse: collapse:** yan yana kenarlıkları tek çizgiye birleştirir, aradaki boşluk kaybolur.

**padding:** hücrelere nefes payı verir, okunurluk artar.

**Hizalama:** metni sola, sayıları sağa hizala; :nth-child ile belirli sütunları seç.

**Zebra deseni:** :nth-child(odd/even) ile satırlara sırayla renk ver; göz satırı kolay takip eder.

**caption-side:** tablo açıklamasını alta taşır.

# 02

# Metin stili

CSS text styling

4 ders

- Fundamental text and font styling
- Styling lists
- Styling links
- Web fonts

font-family: "Helvetica", "Arial", sans-serif;

# Font temelleri: aile, yığın ve boyut

## Font ailesi ve yığını

Tarayıcı sadece kullanıcının bilgisayarında kurulu fontu kullanabilir. Bu yüzden virgülle bir yığın verilir; ilk bulunan kullanılır. Her zaman genel bir aileyle bitir.

```
p { font-family: "Trebuchet MS",
    "Verdana", sans-serif; }
```

Web-safe: Arial, Courier New, Georgia, Times New Roman, Trebuchet MS, Verdana. Genel aileler: serif, sans-serif, monospace, cursive, fantasy. Boşluklu adları tırnakla, genel adları tırnaksız yaz.

## Font boyutu

px mutlak; em ebeveyne göre ve iç içe katlanır; rem köke göre ve hesabı kolaydır. Kök varsayılan 16px, h1 varsayılan 2em = 32px. MDN tavsiyesi: mümkünse rem kullan.

```
html { font-size: 10px; }
h1 { font-size: 5rem; }   /* 50px */
p  { font-size: 1.5rem; } /* 15px */
```

line-height: 1.6;

# Font stilleri ve metin düzeni

| Özellik | Ne yapar? | Örnek |
|---|---|---|
| font-style | italik / eğik | italic |
| font-weight | kalınlık (100–900) | bold |
| text-transform | büyük/küçük harf | uppercase |
| text-decoration | alt/üst çizgi, üstü çizili | none |
| text-shadow | x, y, bulanıklık, renk | 4px 4px 5px red |
| text-align | hizalama | justify |
| line-height | satır yüksekliği | 1.6 |
| letter / word-spacing | harf ve kelime arası | 4px |

**font kısayolu**

```
font: italic bold
  3em/1.5 "Helvetica",
  sans-serif;
```

≈ **Gündelik hayatta:** line-height, defterdeki satır aralığı gibidir: çok sıkışık okunmaz, çok açık kopuk durur.

LoVe FeArs HAte

# Bağlantıları stillemek

**L** Link
`a:link`

**V** Visited
`a:visited`

**F** Focus
`a:focus`

**H** Hover
`a:hover`

**A** Active
`a:active`

## Kurallar

Linkin altını çiz, başka şeyin altını çizme. Hover/focus ve active'te farklı geri bildirim ver. outline'ı başka bir odak göstergesi koymadan kaldırma; cursor'a dokunma.

## İkon ve buton linkler

```
a[href^="http"]::after {
  content: ""; width: 0.8em; }
.container { display: flex; }
a { flex: 1; }
```

≈ **Gündelik hayatta:** trafik ışığı gibi: link her durumda farklı renk verir ki kullanıcı nerede olduğunu bilsin.

# 03

# CSS düzeni

CSS layout

6 ders

- Introduction
- Floats
- Positioning
- CSS grid layout
- Responsive web design
- Media queries

normal flow

# Düzene giriş: normal akış

## Normal akış (normal flow)

- CSS değiştirmediği sürece tarayıcı öğeleri kaynak sırasına göre dizer
- Blok öğeler blok ekseninde (yazı yönüne göre) alt alta, margin'le ayrılarak dizilir
- Satır içi öğeler yer oldukça aynı satırda durur, sığmayan alt satıra kayar
- Dikeyde margin'ler çöker
- İyi yapılandırılmış HTML her ekranda ve ekran okuyucuda okunur; düzene onunla çalışarak başla

## Akışı değiştiren araçlar

**display:** block, inline, inline-block; ayrıca flex ve grid

**float:** metni bir öğenin etrafına sar

**Flexbox ve Grid:** çocukların yerleşimini yöneten düzen sistemleri

**Responsive tasarım:** @media ile cihaza göre farklı düzen

**Diğer:** tablo düzeni, çok sütunlu düzen

≈ **Gündelik hayatta:** normal akış, yazı yazmak gibidir: kelimeler satırda yan yana, paragraflar alt alta. Düzen araçları ise sayfayı gazete gibi bölmek.

`float: left;  display: flow-root;`

# Float: metni etrafına sarmak

floats.css

```css
.box {
  float: left;
  margin-right: 15px;
  width: 150px;
}
```

```css
.cleared { clear: left; }
.wrapper {
  display: flow-root; }
```

## Bilmen gerekenler

**Amaç:** gazetedeki gibi resmin etrafına metni sarmak.

**Margin:** float'a margin vererek metni itersin; metne margin vermek işe yaramaz.

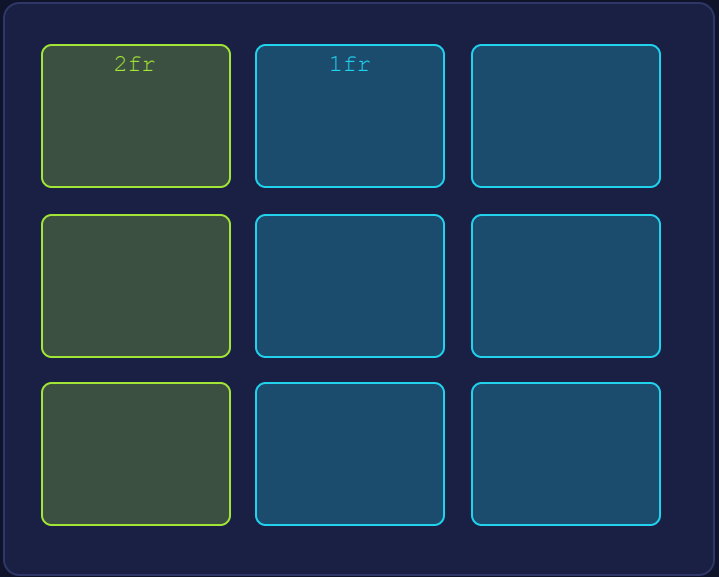
**clear:** left, right, both ile sonraki öğenin float'ın yanına çıkmasını engelle.

**flow-root:** kapsayıcıyı içindeki float'ları sarmaya zorlar; hack gerektirmez.

**Eski yöntem:** float ile sayfa düzeni kurmak artık eski bir tekniktir; bunun için Flexbox ve Grid var.

grid-template-columns: repeat(3, 1fr);

# Grid 1: iki boyutlu düzen




grid.css

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
  grid-auto-rows: minmax(50px, auto);
  gap: 20px; }
```

**fr:** kullanılabilir alanı oranla böler.

**gap:** satır/sütun arası boşluk (gutter).

**repeat():** repeat(3, 1fr) = 1fr 1fr 1fr (tekrarlama)

**Açık / örtük ızgara:** template ile tanımladığın açık; taşan içerik için otomatik oluşan örtük (grid-auto-rows).

**minmax() + auto-fit:** sığdığı kadar sütun.

≈ **Gündelik hayatta:** grid, satranç tahtası ya da Excel sayfası gibidir: hem satırlar hem sütunlar aynı anda kontrol edilir.

grid-template-areas: "header header" …

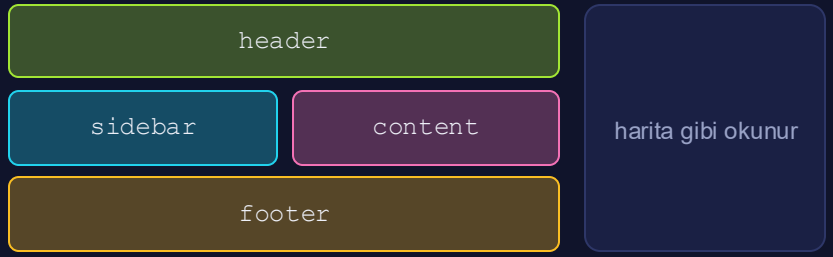
# Grid 2: öğeleri ızgaraya yerleştirmek

cizgiler.css

```
header { grid-column: 1 / 3; grid-row: 1; }
aside  { grid-column: 1;     grid-row: 2; }
main   { grid-column: 2;     grid-row: 2; }
footer { grid-column: 1 / -1; }
```

Çizgi tabanlı yerleşim: çizgiler 1'den başlar, yazı yönünü izler. start / end yazılır; -1 açık ızgaranın son çizgisidir.

```
.container { grid-template-areas:
  "header header"
  "sidebar content"
  "footer footer"; }
```




**Alan kuralları:** her hücre dolu olmalı; . boş hücre; alanlar dikdörtgen olmalı (L şekli yok); aynı ad iki ayrı yerde olamaz.

**İç içe grid ve subgrid:** grid içindeki öğe de grid olabilir; grid-template-columns: subgrid ebeveynin izlerini devralır.

**Framework gerekmez:** repeat(12, 1fr) ile 12 sütunlu düzen yerleşik CSS'te var.

≈ **Gündelik hayatta:** sinema bileti gibi: “3. sıra, 1'den 3'e kadar koltuklar” (çizgiler) ya da “VIP bölümü” (alan adı).

<meta name="viewport" content="width=device-width">

# Duyarlı (responsive) web tasarımı

## Bir teknoloji değil, yaklaşım

2010'da Ethan Marcotte adını koydu: akışkan ızgaralar, akışkan resimler (max-width: 100%) ve media queries. Bugün Flexbox ve Grid kendiliğinden duyarlıdır; mobile özel ayrı site yerine tek esnek düzen.

```
@media screen and (width >= 600px) {
  .wrapper { display: grid;
    grid-template-columns: 1fr 2fr; }
}
```

**Resimler:** img, picture, video { max-width: 100%; }; büyük resmi küçültmek yerine srcset / <picture> ile uygun dosyayı sun.

**Tipografi:** media query ile başlığı büyüt; ya da font-size: calc(1.5rem + 4vw).

**Uyarı:** metni sadece vw ile boyutlandırma: kullanıcı yakınlaştıramaz.

**Viewport meta:** olmazsa mobil tarayıcı sayfayı ~980px sanıp uzaklaştırır, dar ekran kuralların hiç çalışmaz. Her zaman head'e ekle.

≈ **Gündelik hayatta:** su gibi: hangi bardağa koyarsan onun şeklini alır. Duyarlı sayfa da telefon, tablet, ekran fark etmeden kabına uyar.

`@media (30em <= width <= 50em)`

# Medya sorguları ve mobil öncelikli tasarım

media.css

```
@media media-type and (kural) {
  /* koşul sağlanınca */
}
@media print { body { font-size: 12pt; } }
```

**Türler:** all (varsayılan), screen, print

**Özellikler:** width (min-/max- ya da aralık söz dizimi), orientation, hover, pointer (fine / coarse)

**Mantık:** and ile birleştir, virgül = veya, not ile tersine çevir

## Mobil öncelikli (mobile first)



tek sütun | 40em: 2 sütun | 70em: 3 sütun

En küçük ekranla, tek sütunla başla; ekran büyüdükçe sütun ekle. Kırılma noktalarını cihazlara göre değil, içeriğin bozulduğu yere koy ve em kullan. auto-fill + minmax ile bazen hiç sorguya gerek kalmaz.

≈ **Gündelik hayatta:** mobil öncelikli tasarım, önce küçük bavulu hazırlayıp yer açıldıkça eşya eklemek gibi; büyükten küçüğe sıkıştırmaktan kolaydır.

# Ek kılavuzlar: bir sonraki adım

### Gelişmiş efektler

box-shadow, filter, karışım modları

### Cascade layers

@layer ile kuralları katmanlara ayırma

### Çok yönlü metin

writing-mode ve mantıksal özellikler

### CSS'i düzenlemek

yorumlar, stil rehberi, metodolojiler

### Çok sütunlu düzen

column-count, column-width

### Pratik konumlandırma

sekmeli kutu, kayan panel örnekleri

### Eski düzen yöntemleri

float tabanlı ızgara sistemleri

### Eski tarayıcılar

@supports ile özellik sorguları

■ CSS styling basics ek kılavuzları ■ CSS layout ek kılavuzları

≈ **Gündelik hayatta:** ana yemekten sonraki tatlı menüsü: zorunlu değil, ama merak edene yeni lezzetler sunar.

/* özet */

# Özet: bugün ne öğrendik?

## Kural

seçici { özellik: değer; } · harici dosya tercih edilir

## Seçiciler

tür, sınıf, id, öznitelik, sözde, birleştirici

## Box model

content, padding, border, margin · border-box

## Çakışmalar

cascade, özgüllük, kalıtım · !important'tan kaçın

## Metin

font yığını, rem, line-height, LoVe FeArs HAte, @font-face

## Düzen

normal akış, float, position, flex (1B), grid (2B), mobil öncelikli

≈ **Gündelik hayatta:** HTML iskelet, CSS kıyafet ve duruş: aynı insan farklı kıyafetle bambaşka görünür, ama iskelet aynı kalır.

KAPANIŞ

SORU-CEVAP ZAMANI

# Teşekkürler!

Sorularınızı bekliyoruz.

KCŞ A.Ş. | CSS styling basics · CSS text styling · CSS layout

Kaynak: MDN Web Docs · Learn web development › Core

son.css

```css
.sunum {
  display: flex;
  align-items: center;
}
.sorular:hover {
  cursor: pointer;
}
```

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs, Vue.js, Three.js vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.